

Trabajo Práctico Integrador – Parte 1

"BiblioTech" – Catálogo y gestor de libros

Contexto

Van a desarrollar una aplicación web de una sola página (SPA simple, sin frameworks) para gestionar el catálogo de una biblioteca. La app debe permitir visualizar libros, agregarlos, marcarlos como favoritos y/o prestados, filtrarlos y buscarlos. Todo manipulando el DOM dinámicamente a partir de datos en JavaScript, sin recargar la página.

Objetivos de aprendizaje

- Integrar HTML semántico, CSS y JS en un proyecto único y cohesivo.
- Practicar manipulación del DOM: crear, actualizar y eliminar elementos dinámicamente a partir de datos.
- Aplicar manejo de eventos de forma prolija, evitando código repetido.
- Usar lógica booleana y operador ternario en la lógica de filtrado/renderizado condicional.
- Reforzar CSS con selectores avanzados, pseudoclases, pseudoelementos y layouts con flex/grid.
- Practicar organización de código JS (funciones puras para renderizar, separación de responsabilidades).

Requisitos funcionales

1. Estructura de datos

- Un arreglo de objetos **libros**, donde cada libro tiene al menos: **id**, **titulo**, **autor**, **genero**, **año**, **disponible** (boolean), **favorito** (boolean).
- Puede estar hardcodeado en un archivo **data.js** (mínimo 8-10 libros para que se note el filtrado).

2. Renderizado dinámico

- El listado de libros se muestra en pantalla generado **por JS**, no escrito a mano en el HTML (usar **createElement**, **innerHTML** controlado, o template strings; a elección, pero justificado).
- Cada libro se muestra como una "tarjeta" con su información y botones de acción.

3. Funcionalidades obligatorias

- **Agregar libro**: un formulario (con validación básica: campos no vacíos, año numérico y razonable) que agrega un nuevo libro al arreglo y lo renderiza sin recargar la página.
- **Marcar como favorito**: un botón/ícono por libro que alterna su estado **favorito** y refleja el cambio visualmente (ej: clase CSS distinta, ícono lleno/vacío).
- **Marcar disponible/prestado**: botón que alterna **disponible** y cambia el estilo de la tarjeta (ej: opacidad reducida o etiqueta "Prestado").
- **Eliminar libro**: quitarlo del arreglo y del DOM.
- **Filtrar por género**: un **<select>** que filtra el listado según género (incluye opción "Todos").
- **Buscar por título/autor**: un input de texto que filtra en tiempo real (evento **input**) usando lógica booleana (**includes**, **toLowerCase**, etc.).
- **Mostrar solo favoritos**: un checkbox (normal o switch) que, activado, muestra solo los libros favoritos (usar operador ternario en la lógica de filtrado).

4. Contador dinámico

- Mostrar en algún lugar visible cuántos libros hay en total y cuántos se están mostrando según el filtro actual (esto se actualiza cada vez que se renderiza).

Requisitos técnicos por tecnología

HTML

- Uso de etiquetas semánticas: **<header>**, **<main>**, **<section>**, **<footer>**, **<form>**, etc.
- Formulario con los **<input>** con **type** apropiado (**text**, **number**, **select**) y **label** asociado a cada campo.

CSS

- Layout principal con **Grid** (para la disposición general: header, filtros, listado) y **Flexbox** dentro de cada tarjeta (para alinear info y botones).
- Al menos 2 pseudoclasas (**:hover**, **:focus**, **:nth-child**, **:disabled**, etc.) y 1 pseudoelemento (**:before**, **:after**).
- Uso correcto de especificidad: evitar **!important**; si lo necesitan en algún punto, deben poder explicar por qué no lo resolvieron de otra forma.
- Responsive mínimo: que no se rompa en una pantalla angosta (no hace falta mobile-first perfecto, pero sí que el grid/flex se adapten con **flex-wrap** o **grid-template-columns** con **auto-fit/minmax**).

JavaScript

- Separar el código en al menos: **data.js** (datos), **render.js** o similar (funciones de renderizado), **app.js** (listeners y lógica).
- Funciones con nombres claros y responsabilidad única.
- Uso de métodos de arreglos (**filter**, **map**, **find**, **some**, etc. Todo lo visto hasta ahora).
- Uso de al menos un operador ternario y lógica booleana combinada (**&&**, **||**) en alguna condición de filtrado o render.
- Nada de **onclick="..."** en el HTML: todos los eventos se agregan desde JS con **addEventListener**.

Funcionalidades opcionales

- Ordenar el listado por año o por título (ascendente/descendente).
- Persistencia simple usando **localStorage**.
- Modo oscuro con un toggle (para practicar variables CSS).
- Animaciones simples con CSS (**transition** al agregar/eliminar tarjetas).

Entrega

- Repositorio en **GitHub**
- Estructura de carpetas prolija: **index.html**, **/css**, **/js** (con los archivos separados como se pidió).
- El repo debe tener un **README.md** con: su nombre y qué funcionalidades opcionales implementaron (si hicieron alguna).
- Commits con mensajes descriptivos; no un solo commit con todo el código. Se valora ver el proceso.

